Streaming Messages

Copy page

When creating a Message, you can set `"stream": true` to incrementally stream the response using server-sent events (SSE).

Streaming with SDKs

Our Python and TypeScript SDKs offer multiple ways of streaming. The Python SDK allows both sync and async streams. See the documentation in each SDK for details.

Python

```python
import anthropic

client = anthropic.Anthropic()

with client.messages.stream(
    max_tokens=1024,
    messages=[{"role": "user", "content": "Hello"}],
    model="claude-sonnet-4-5",
) as stream:
  for text in stream.text_stream:
      print(text, end="", flush=True)
```

Event types

Each server-sent event includes a named event type and associated JSON data. Each event will use an SSE event name (e.g. `event: message_stop`), and include the matching event `type` in its data.

Each stream uses the following event flow:

1. `message_start`: contains a `Message` object with empty `content`.
2. A series of content blocks, each of which have a `content_block_start`, one or r
   `content_block_delta` events, and a `content_block_stop` event. Each content

4. A final `message_stop` event.

> ⚠ The token counts shown in the `usage` field of the `message_delta` event are *cumulative*.

Ping events

Event streams may also include any number of `ping` events.

Error events

We may occasionally send errors in the event stream. For example, during periods of high usage, you may receive an `overloaded_error`, which would normally correspond to an HTTP 529 in a non-streaming context:

Example error

```
event: error
data: {"type": "error", "error": {"type": "overloaded_error", "message": "Overloaded"}}
```

Other events

In accordance with our versioning policy, we may add new event types, and your code should handle unknown event types gracefully.

Content block delta types

Each `content_block_delta` event contains a `delta` of a type that updates the `content` block at a given `index`.

Text delta

A `text` content block delta looks like:

Text delta

```
event: content_block_delta
data: {"type": "content_block_delta","index": 0,"delta": {"type": "text_delta", "text": "
```

The deltas for `tool_use` content blocks correspond to updates for the `input` field of the block. To support maximum granularity, the deltas are *partial JSON strings*, whereas the final `tool_use.input` is always an *object*.

You can accumulate the string deltas and parse the JSON once you receive a `content_block_stop` event, by using a library like Pydantic to do partial JSON parsing, or by using our SDKs, which provide helpers to access parsed incremental values.

A `tool_use` content block delta looks like:

Input JSON delta

```
event: content_block_delta
data: {"type": "content_block_delta","index": 1,"delta": {"type": "input_json_delta","par
```

Note: Our current models only support emitting one complete key and value property from `input` at a time. As such, when using tools, there may be delays between streaming events while the model is working. Once an `input` key and value are accumulated, we emit them as multiple `content_block_delta` events with chunked partial json so that the format can automatically support finer granularity in future models.

Thinking delta

When using extended thinking with streaming enabled, you'll receive thinking content via `thinking_delta` events. These deltas correspond to the `thinking` field of the `thinking` content blocks.

For thinking content, a special `signature_delta` event is sent just before the `content_block_stop` event. This signature is used to verify the integrity of the thinking block.

A typical thinking delta looks like:

Thinking delta

```
event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "thinking_delta", "th
```

The signature delta looks like:

```
event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "signature_delta", "s
```

Full HTTP Stream response

We strongly recommend that you use our client SDKs when using streaming mode. However, if you are building a direct API integration, you will need to handle these events yourself.

A stream response is comprised of:

1. A `message_start` event
2. Potentially multiple content blocks, each of which contains:
   - A `content_block_start` event
   - Potentially multiple `content_block_delta` events
   - A `content_block_stop` event
3. A `message_delta` event
4. A `message_stop` event

There may be `ping` events dispersed throughout the response as well. See Event types for more details on the format.

Basic streaming request

Shell

```shell
curl https://api.anthropic.com/v1/messages \
     --header "anthropic-version: 2023-06-01" \
     --header "content-type: application/json" \
     --header "x-api-key: $ANTHROPIC_API_KEY" \
     --data \
'{
  "model": "claude-sonnet-4-5",
  "messages": [{"role": "user", "content": "Hello"}],
  "max_tokens": 256,
  "stream": true
}'
```

```
event: message_start
data: {"type": "message_start", "message": {"id": "msg_1nZdL29xx5MUA1yADyHTEsnR8uuvGzszyY

event: content_block_start
data: {"type": "content_block_start", "index": 0, "content_block": {"type": "text", "text

event: ping
data: {"type": "ping"}

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "text_delta", "text":

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "text_delta", "text":

event: content_block_stop
data: {"type": "content_block_stop", "index": 0}

event: message_delta
data: {"type": "message_delta", "delta": {"stop_reason": "end_turn", "stop_sequence":null

event: message_stop
data: {"type": "message_stop"}
```

Streaming request with tool use

> Tool use now supports fine-grained streaming for parameter values as a beta feature. For more details, see Fine-grained tool streaming.

In this request, we ask Claude to use a tool to tell us the weather.

Shell

```
    -H "x-api-key: $ANTHROPIC_API_KEY" \
    -H "anthropic-version: 2023-06-01" \
    -d '{
      "model": "claude-sonnet-4-5",
      "max_tokens": 1024,
      "tools": [
        {
          "name": "get_weather",
          "description": "Get the current weather in a given location",
          "input_schema": {
            "type": "object",
            "properties": {
              "location": {
                "type": "string",
                "description": "The city and state, e.g. San Francisco, CA"
              }
            },
            "required": ["location"]
          }
        }
      ],
      "tool_choice": {"type": "any"},
      "messages": [
        {
          "role": "user",
          "content": "What is the weather like in San Francisco?"
        }
      ],
      "stream": true
    }'
```

Response

```
event: content_block_start
data: {"type":"content_block_start","index":0,"content_block":{"type":"text","text":""}}

event: ping
data: {"type": "ping"}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":"Okay"}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":","}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" let"}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":"'s"}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" check

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" the"}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" weath

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" for"}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" San"}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" Franc

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":","}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" CA"}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":":"}}

event: content_block_stop
data: {"type":"content_block_stop","index":0}
```

```
event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_stop
data: {"type":"content_block_stop","index":1}

event: message_delta
data: {"type":"message_delta","delta":{"stop_reason":"tool_use","stop_sequence":null},"usa

event: message_stop
data: {"type":"message_stop"}
```

Streaming request with extended thinking

In this request, we enable extended thinking with streaming to see Claude's step-by-step reasoning.

Shell

```
     --header "anthropic-version: 2023-06-01" \
     --header "content-type: application/json" \
     --data \
'{
    "model": "claude-sonnet-4-5",
    "max_tokens": 20000,
    "stream": true,
    "thinking": {
        "type": "enabled",
        "budget_tokens": 16000
    },
    "messages": [
        {
            "role": "user",
            "content": "What is 27 * 453?"
        }
    ]
}'
```

Response

```
event: content_block_start
data: {"type": "content_block_start", "index": 0, "content_block": {"type": "thinking", "

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "thinking_delta", "th

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "thinking_delta", "th

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "thinking_delta", "th

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "thinking_delta", "th

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "thinking_delta", "th

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "thinking_delta", "th

event: content_block_delta
data: {"type": "content_block_delta", "index": 0, "delta": {"type": "signature_delta", "s

event: content_block_stop
data: {"type": "content_block_stop", "index": 0}

event: content_block_start
data: {"type": "content_block_start", "index": 1, "content_block": {"type": "text", "text"

event: content_block_delta
data: {"type": "content_block_delta", "index": 1, "delta": {"type": "text_delta", "text":

event: content_block_stop
data: {"type": "content_block_stop", "index": 1}

event: message_delta
data: {"type": "message_delta", "delta": {"stop_reason": "end_turn", "stop_sequence": nul

event: message_stop
data: {"type": "message_stop"}
```

Streaming request with web search tool use

Shell

```shell
curl https://api.anthropic.com/v1/messages \
     --header "x-api-key: $ANTHROPIC_API_KEY" \
     --header "anthropic-version: 2023-06-01" \
     --header "content-type: application/json" \
     --data \
'{
    "model": "claude-sonnet-4-5",
    "max_tokens": 1024,
    "stream": true,
    "tools": [
        {
            "type": "web_search_20250305",
            "name": "web_search",
            "max_uses": 5
        }
    ],
    "messages": [
        {
            "role": "user",
            "content": "What is the weather like in New York City today?"
        }
    ]
}'
```

Response

```
event: content_block_start
data: {"type":"content_block_start","index":0,"content_block":{"type":"text","text":""}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":"I'll cl

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":" the cu

event: ping
data: {"type": "ping"}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"text_delta","text":"."}}

event: content_block_stop
data: {"type":"content_block_stop","index":0}

event: content_block_start
data: {"type":"content_block_start","index":1,"content_block":{"type":"server_tool_use","

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":{"type":"input_json_delta","partial_

event: content_block_stop
data: {"type":"content_block_stop","index":1 }

event: content_block_start
data: {"type":"content_block_start","index":2,"content_block":{"type":"web_search_tool_res
```

```
event: content_block_start
data: {"type":"content_block_start","index":3,"content_block":{"type":"text","text":""}}

event: content_block_delta
data: {"type":"content_block_delta","index":3,"delta":{"type":"text_delta","text":"Here's
event: content_block_delta
data: {"type":"content_block_delta","index":3,"delta":{"type":"text_delta","text":" City:
event: content_block_delta
data: {"type":"content_block_delta","index":3,"delta":{"type":"text_delta","text":" in New
event: content_block_delta
data: {"type":"content_block_delta","index":3,"delta":{"type":"text_delta","text":"\n\n"}}
event: content_block_stop
data: {"type":"content_block_stop","index":3}
event: message_delta
data: {"type":"message_delta","delta":{"stop_reason":"end_turn","stop_sequence":null},"usa

event: message_stop
data: {"type":"message_stop"}
```

1. **Capture the partial response**: Save all content that was successfully received before the error occurred
2. **Construct a continuation request**: Create a new API request that includes the partial assistant response as the beginning of a new assistant message
3. **Resume streaming**: Continue receiving the rest of the response from where it was interrupted

Error recovery best practices

1. **Use SDK features**: Leverage the SDK's built-in message accumulation and error handling capabilities
2. **Handle content types**: Be aware that messages can contain multiple content blocks (`text` , `tool_use` , `thinking`). Tool use and extended thinking blocks cannot be partially recovered. You can resume streaming from the most recent text block.